

Cursor.AI를 활용한 소프트웨어 개발 ② Test

Dual-Track TDD · C2C × ARRR

RED

GREEN

REFACTOR

불변식·계약 ID → 테스트 → 구현으로 끊임 없이 잇는 추적 가능한 개발

이 강의가 끝나면 할 수 있는 것



추적성(C2C)

불변식·Error Contract에 ID를 부여하고 테스트·구현까지 잇는다



Dual-Track / ECB

Boundary(UI)와 Entity(Domain)를 분리해 각 트랙에서 사이클을 돈다



ARRR 리듬

Ask(RED)→Respond(GREEN)→Refine(REFACTOR)→Repeat



Augmented Coding

AI에 위임하되 주도권을 쥐고 '씨앗을 먹는' 설계 후퇴를 막는다

THE PROBLEM

이 워크북이 바꾸는 단 하나

기존: 스켈레톤 바로 뒤에 완성 코드를 노출 → '이 코드가 어느 요구·개념에서 나왔는가'를 역추적할 수 없다 (C2C 단절).

기존 방식

목표 → 스켈레톤 → **완성 코드**
코드 ↔ 요구 사이 연결고리 끊김
"이 줄은 왜 있지?" 에 답할 수 없음



이 과정

불변식·계약 ID → 테스트 → 구현
Discovery → Delivery(ARRR)
구현 줄마다 계약 ID 주석
어느 코드든 역추적 가능

TDD — RED → GREEN → REFACTOR

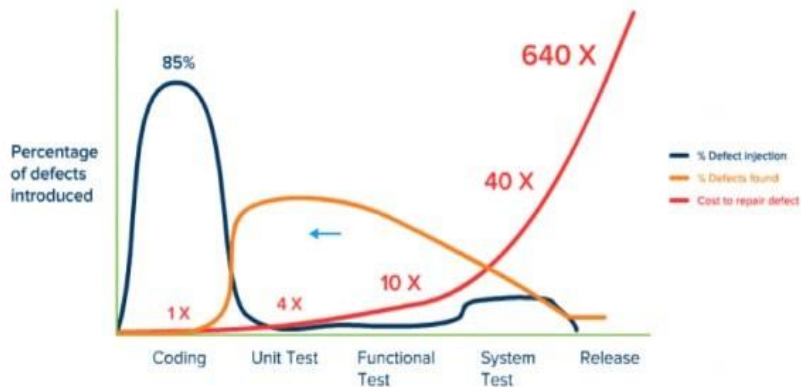
테스트를 먼저 쓰고 통과시키는 개발. 한 사이클은 두 단계가 아니라 세 단계다 (켄트 벡, 2002).



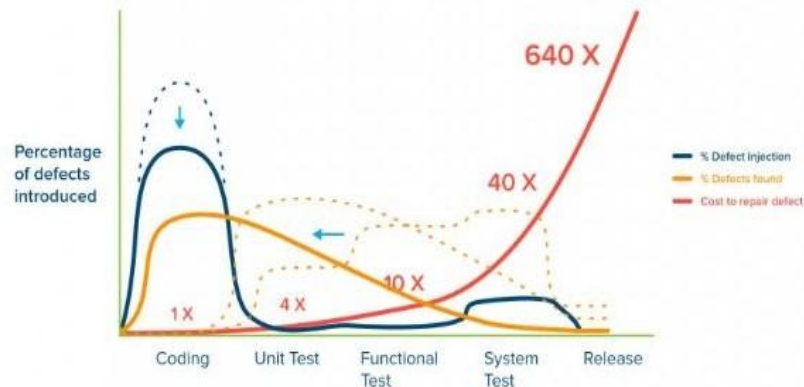
REFACTOR를 빼면 TDD가 아니라 '테스트 먼저 쓰기'에 그친다.

결함 비용 — Shift-Left Testing

결함 발견 시점이 늦을수록 수정 비용이 기하급수적으로 증가 → 초기 테스트(Shift-Left)가 핵심



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

리팩터링 & Tidy First

리팩터링 = 겉보기 동작은 그대로 두고 내부 구조만 개선 (마틴 파울러). 전제는 '테스트'다.

구조적 변경 (Structural)

이름 바꾸기 · 함수 추출 · 코드 이동
동작은 그대로
→ refactor 커밋

동작적 변경 (Behavioral)

기능 추가 · 수정
동작이 달라짐
→ feat / fix 커밋

Tidy First 규칙: 둘을 한 커밋에 섞지 말 것 · 둘 다 필요하면 구조 먼저 · 전후로 테스트 확인

Augmented Coding — “씨앗을 먹지 마라”

Vibe Coding

돌아가면 OK, 결과를 거의 안 본다
관심: 동작만 / 말기고 기다린다

Augmented Coding

개발자가 주도권 · 결과에 책임
동작 + 테스트 + 설계 / 중간 감시·개입



AI가 걸길로 새는 경고 신호

- ① 요청하지 않은 기능 (합리적 다음 단계라도 = 과잉 구현)
- ② 테스트를 비활성화·삭제·수정해 '속이는' 정황

Concept-to-Code 4계층

'무엇이 절대 바뀌면 안 되는가(불변식)'에서 출발해 코드까지 — 검증은 위에서 아래로 내려간다.

Layer 1	Invariant (불변식)	무엇이 절대 바뀌면 안 되는가	수학적 진실
Layer 2	UI Contract Test	UX가 올바르게 반응하는가	Boundary
Layer 3	Domain Test	비즈니스 규칙이 작동하는가	Control
Layer 4	Implementation Unit	단위 알고리즘이 정확한가	Entity

두 트랙과 ECB

Track A — UI / Boundary

입력 계약 위반을 Boundary에서 차단
Mock으로 Domain 격리
`tests/boundary/`

Track B — Domain / Logic

도메인 알고리즘을 순수 함수로 검증
Mock 없이 실제 Entity 호출(외부 I/O만 mock)
`tests/entity/`

ECB import 규칙



Boundary

입력 검증·출력 형식화 (도메인 알고리즘 금지)



Control

유스케이스 조율 (I/O 검증·UI 문자열 금지)



Entity

순수 로직·불변식
(Boundary/Control import 금지)

ARRR ≡ RED·GREEN·REFACTOR

ARRR는 새 이론이 아니라 Dual-Track TDD의 '실행 리듬'이다.

A — Ask	테스트로 시스템에 질문한다	RED	FAIL
R — Respond	통과하는 최소 코드로 답한다	GREEN	PASS
R — Refine	중복 제거·이름 정리로 다듬는다	REFACTOR	계약 불변
R — Repeat	다음 질문으로 계속한다	다음 RED	회귀 확인

8단계 골격 — Discovery → Delivery

Discovery (코드 짜기 전)

- 1 목표 — 보장 + ECB·C2C·트랙
- 2 PRD — '무엇이 실패인가' L0~L3
- 3 불변식·Error Contract에 ID 부여
- 4 스켈레톤 — 시그니처+TODO (완성 코드 감춤)

Delivery — ARRR

- 5 Ask (RED) — ID마다 실패 테스트, src/ 금지
- 6 Respond (GREEN) — 최소 구현 + ID 주석
- 7 Refine (REFACTOR) — SSOT 추출
- 8 Repeat — 다음 ID + 회귀, 최소 3회

작업 환경 — 단축키 · 규칙 파일

⌘/Ctrl + L

Chat — 묻고 답하기

⌘/Ctrl + I

Agent/Composer — 자율 편집·실행

⌘/Ctrl + K

선택 영역 인라인 편집

@

@파일·@Web·@Git 으로 Context 지정

규칙은 코드처럼 버전관리한다

규칙 2종

AGENTS.md

루트의 평문 마크다운 '지도' (짧게!)

.cursor/rules/*.mdc

머리말(alwaysApply·globs)로 적용 시점 제어

RED: tests/ 만 수정

GREEN: 최소 구현 + 계약 ID 주석

REFACTOR: 전부 통과 후 정리

테스트 우회·과잉 구현 금지

하네스 & 폴더 구조 (ECB 정렬)

```
cursor-tdd-cart/
├── AGENTS.md                # 항상 적용 규칙(지도)
├── .cursor/rules/tdd.mdc
├── pytest.ini · conftest.py
├── src/
│   ├── cart.py             # Entity (Track B)
│   └── app.py              # Boundary (Track A)
├── tests/
│   ├── entity/             # INV-* 검증
│   └── boundary/          # E-*, UC-* 검증
```

폴더가 곧 추적이다

테스트를 계층별로 나누면
'어느 계층의 계약을 검증하나'
를 폴더만 봐도 알 수 있다.

테스트 명령:

```
pytest -q
```

WORKED EXAMPLE

장바구니 할인 계산기 — 계약 ID

INV-1

subtotal = $\Sigma(\text{price} \times \text{qty})$

Entity

INV-2

$\geq 50000 \rightarrow \times 0.9$ 반올림 / 미만 그대로 (경계 포함)

Entity

INV-3

문턱할인 후 VIP면 $\times 0.95$, 순서 고정

Entity

INV-4

$0 \leq \text{final_total} \leq \text{subtotal}$ (할인이 금액을 늘리지 않음)

Entity

E-1

items=None $\rightarrow$ TypeError

Boundary

E-2

price/qty 음수 $\rightarrow$ ValueError(인덱스 포함)

Boundary

실패 조건(L0~L3)과 스켈레톤

L0

None인데 조용히 0 반환 → 예외 필수

L1

합계 50000인데 할인 미적용 (>= 위반)

L2

VIP 누락 / 문턱→VIP 순서 뒤바뀜

L3

최종 금액이 합계보다 커짐

기능 목록이 아니라 '무엇이 실패인가'를 적는다

```
THRESHOLD = 50000    # INV-2 (SSOT)
```

```
def subtotal(items):
```

```
    # TODO(E-1): None → TypeError
```

```
    # TODO(E-2): 음수 → ValueError
```

```
    # TODO(INV-1):  $\Sigma price*qty$ 
```

```
    raise NotImplementedError
```

```
def final_total(items, is_vip=False):
```

```
    # TODO(INV-3): 문턱→VIP 순서
```

```
    # TODO(INV-4): subtotal 이하 보장
```

```
    raise NotImplementedError
```

subtotal — Ask → Respond

RED

```
# tests/entity/test_cart.py
from src.cart import subtotal

def test_subtotal_sums(): # INV-1
    items=[{'price':1000,'qty':3},
            {'price':2000,'qty':2}]
    assert subtotal(items)==7000
```

GREEN

```
def subtotal(items):
    if items is None: # E-1
        raise TypeError(...)
    for i,it in enumerate(items): # E-2
        if it['price']<0 or it['qty']<0:
            raise ValueError(f'... {i}')
    return sum(it['price']*it['qty'] # INV-1
               for it in items)
```

threshold — 경계값 & SSOT

```
# 경계값 두 개로 '이상(>=)' 못박기
assert apply_threshold_discount(50000)==45000
assert apply_threshold_discount(49999)==49999
```

```
def apply_threshold_discount(amount):
    if amount >= THRESHOLD: # INV-2
        return round(amount*THRESHOLD_RATE)
    return amount
```



자주 하는 실수

amount > 50000 으로 짜면 경계(50000) 실패.
요구는 '이상(>=)'. AI가 테스트를 바꿔 통과시키려 하면 즉시 막기.

REFACTOR — SSOT 추출

0.9 → THRESHOLD_RATE 상수. 동작 불변이므로 refactor
커밋으로 분리, 전후로 pytest -q.

final_total — 순서 & 수학적 불변

```
def final_total(items, is_vip=False):  
    amount = apply_threshold_discount(  
        subtotal(items))    # INV-3 문턱 먼저  
    if is_vip:  
        amount = round(amount*VIP_RATE) # INV-3 그다음 VIP  
    return amount    # INV-4 subtotal 이하 보장
```

```
# INV-4: 여러 입력에서  $0 \leq \text{final} \leq \text{subtotal}$   
for items,vip in samples:  
    assert  $0 \leq \text{final\_total}(\text{items},\text{vip}) \leq \text{subtotal}(\text{items})$ 
```

두 계약 한 번에

INV-3

적용 순서를 고정(문턱→VIP)

INV-4

기존 함수 재사용이 '할인만 적용 → 금액이 늘 수 없음'을 자연스럽게 보장

C2C 추적성 매트릭스

계약 ID ↔ 테스트 ↔ 구현이 1:1로 이어지면 C2C가 보장된 것이다.

계약 ID	검증 테스트	구현 위치
INV-1	entity::test_subtotal_sums	subtotal() return
INV-2	entity::test_threshold_* (경계 2개)	apply_threshold_discount()
INV-3	entity::test_vip_order_and_value	final_total()
INV-4	entity::test_discount_never_increases	final_total() return
E-1	boundary::test_none_items	subtotal() 가드
E-2	boundary::test_negative_qty	subtotal() 루프

UI 트랙 — 계약 & ECB 방향

UC-1

GET / → 200 + 수량·VIP 입력 폼

UC-2

POST /calc → final_total 결과 표시

UE-1

비숫자 qty → 400 + 에러 메시지

ECB 방향 단속

Boundary(app.py)는 Entity(final_total)를 재 사용

Entity는 Flask를 import하지 않음

Boundary 테스트는 '계산이 맞나'가 아니라 '올바른 함수를 부르고 결과를 형식대로 돌려주나'를 본다.

커밋 · Review · 확장↔수축

```
$ git log --oneline
refactor(entity): VIP_RATE SSOT 추출 [REFACTOR]
feat(entity): INV-3·INV-4 구현 [GREEN]
test(entity): INV-3·INV-4 실패 테스트 [RED]
... # RED·GREEN·REFACTOR 절대 한 커밋에 안 섞음
```

확장 ↔ 수축 균형

기능 2~3개 추가(확장)마다 리팩터(수축) 한 번 — '씨앗을 먹지 않는' 리듬

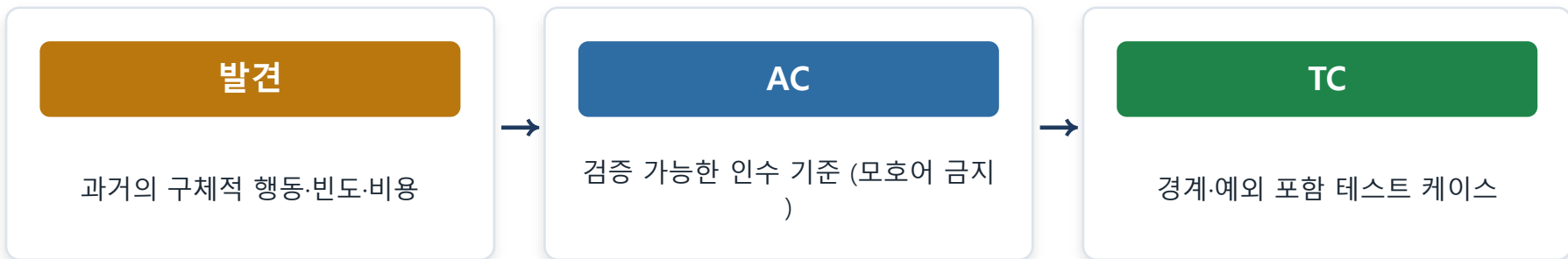


Review (수정 금지)

- ① 구현 줄에 계약 ID 주석 있나
- ② 추적성 매트릭스 빈칸
- ③ 과잉 구현·우회
- ④ Tidy First 커밋 분리
- ⑤ ECB 방향(Entity→Boundary 금지)

올바른 계약은 어디서? — Mom Test

계약(불변식)이 '맞는 것'을 겨누게 하는 상류 도구. AC를 뽑는 도구가 아니라 한 칸 위에 있다.



Mom Test 3원칙: 내 아이디어가 아니라 상대의 삶을 · 미래의 의견이 아니라 과거의 사건을 · 적게 말하고 많이 듣는다

WRAP - UP

어느 코드 한 줄을 보더라도 “이건 어느 계약을 지키려는 거지?”에 답할 수 있다

— 그것이 C2C다.

Discovery

목표·PRD·계약 ID·스켈레톤

Delivery

Ask→Respond→Refine→Repeat

추적성

계약↔테스트↔구현 매트릭스

Cursor.AI · Python 3.12 · pytest · Flask · Git | RED · GREEN · REFACTOR ≡ ARRR